



Bondly

Full-stack social media web app

Project Overview and UI Walkthrough

Bondly is my individual full-stack social media web application. I built it to connect real backend workflows with a usable React interface: registration, OTP verification, authentication, profiles, posts, social interactions, real-time notifications, support handling, and account recovery.

Spring Boot Backend

React Frontend

PostgreSQL

JWT Auth

SSE Realtime

What this document covers

This PDF explains the project from the outside user's point of view and from the developer side. It includes the stack I used, the main feature set, the backend and frontend structure, API areas, security notes, and a screenshot-based UI walkthrough.

2

Main applications

16

Frontend routes

13+

API areas

10

UI screenshots

Project Overview

Application idea

Bondly works like a small social network. A user can create an account, verify it through email OTP, login, update the profile, create posts, react, comment, share, follow users, manage friends, receive notifications, and ask support questions.

The project is not only a UI mockup. The frontend is connected to a Spring Boot REST backend with database persistence, authentication, upload handling, and service-layer business logic.

Main layers

React UI

Pages, navigation, forms, feed, profile, support, notifications.

API Client

Axios and EventSource connect the frontend to backend endpoints.

Spring API

Controllers expose auth, posts, user, support, social, and review routes.

Service Layer

Business logic for OTP, social actions, uploads, realtime events.

Database

PostgreSQL stores users, posts, comments, friends, follows, notifications.

Authentication

Registration uses email OTP verification. Login uses Spring Security, BCrypt, and JWT tokens. Password reset uses a separate OTP flow and a short-lived verification token.

Social features

Posts, image uploads, comments, replies, reactions, shares, followers, following, and friend requests are split into clear backend modules.

Realtime features

Notifications and support updates use Server-Sent Events, so active users can receive updates without manually refreshing the page.

Technology Stack

Backend

- Java 25
- Spring Boot 4.0.2
- Spring Web MVC
- Spring Security
- Spring Data JPA
- PostgreSQL
- JWT / jjwt
- BCrypt
- Java Mail Sender
- Server-Sent Events
- Lombok
- Maven

Frontend

- React 19
- Vite
- React Router
- Axios
- Material UI
- MUI Icons
- Lucide React
- Framer Motion
- Styled Components
- CSS page styling

Why this stack fits the project

Spring Boot gives the backend a structured controller-service-repository style. Spring Security and JWT handle authentication. JPA keeps database work manageable. React and Vite make the frontend fast to build and easy to organize into pages. Material UI, icons, and animation libraries help the interface feel more complete without writing every UI control from zero.

Backend Structure

Controllers

REST endpoints are grouped by feature: auth, user, posts, comments, reactions, shares, follows, friends, notifications, support, reviews, and chatbot.

Services

The service layer keeps the main logic away from controllers. It handles account flows, OTP logic, social actions, uploads, and realtime events.

Repositories

Spring Data repositories handle persistence for users, posts, comments, reactions, shares, follows, friends, notifications, reviews, and support questions.

Feature coverage

Email OTP registration and verification	JWT login with HTTP-only cookie support	BCrypt password hashing
Forgot-password OTP recovery	Profile and cover image upload	Text/image posts and feed
Comments, replies, and delete ownership checks	Multiple reaction types with counts	Post sharing with captions
Followers, following, user search	Friend request workflow	Real-time notifications with SSE
User/admin support flow	Reviews and ratings	Role-based dashboard routing

Frontend Routes and API Areas

Frontend route map

/	Landing page
/signup	Create account
/verify	OTP verification
/login	Login
/forgot-password	Forgot password
/dashboard	Admin dashboard
/home	Main feed
/profile	Profile
/settings	Account settings
/review	Reviews
/supportAdmin	Admin support
/supportUser	User support
/searchUsers	User search
/notifications	Notifications
/friend	Friends
/post	Post/share page

Backend endpoint groups

/auth	Register, login, verify account OTP, resend OTP	/forgotpass	Forgot password OTP and password reset
/user	Profile, settings, uploads, account deletion	/posts	Create posts, delete posts, personal posts, feed
/comments	Comments, replies, delete, counts	/reactions	Add/remove reactions and reaction counts
/shares	Share posts and shared feed	/follow	Follow/unfollow, followers, following, search
/api/friends	Friend requests and friend list	/notifications	Notifications and SSE stream
/support	User questions, admin responses, SSE stream	/reviews	Review CRUD
/api/chat	Rule-based and fuzzy concept chatbot		

Repository Notes

Local configuration

I keep the real `application.yaml` file only on my local machine because it contains values for the database, mail account, and JWT signing secret.

For GitHub, I added `application.example.yaml`. It shows the configuration format without exposing private credentials.

Files I do not commit

Local config files and environment files

Runtime upload folders

Build output such as target and dist

Node modules and log files

Public repository note

The latest project state keeps private config and runtime files out of Git. If any real credentials were pushed before this cleanup, I would change those credentials before using the project publicly.

I also masked personal contact details in the profile screenshot so the UI walkthrough can be shared without exposing private information.